

Research Companion 0.3

The Complete Guide & Critical Analysis

What the tool does, how to use it, what it does well - and where it honestly falls short.

For readers with no prior knowledge of the tool. Every number in this document comes from committed artifacts or measured live runs.

1. What is Research Companion?

Research Companion is an open-source assistant for people writing and evaluating research papers. You give it papers - your own draft and the literature around it - and a team of specialized AI agents reads them, builds a shared knowledge graph, and answers the questions every author faces: **Are my references real? Is my contribution actually novel? Which papers support or threaten my draft, and where? How do I answer this reviewer?**

The one-sentence version: **a team of AI helpers that reads your papers and checks its own answers before showing you.** That last part is the whole design. Most AI tools hand you fluent text and leave you to wonder what is real. Research Companion is built the other way around: every verdict carries evidence you can check, every quote is verified against the source text, and when something cannot be verified the tool says so on the screen instead of hiding it.

It runs entirely on your machine (a local store in `~/research-companion`), talks to an AI provider you configure (Anthropic or OpenAI) only when a task needs it, and costs pennies per paper. It is MIT-licensed, has 1,576 automated tests, and ships zero-key offline demos so you can see everything work before configuring any account.

Who it is for

- **Researchers and students** preparing a submission: catch fabricated or wrong references, stress-test the novelty claim, and see how new papers bear on the draft.
- **Anyone managing a growing pile of PDFs:** turn a folder of papers into a searchable, queryable knowledge graph with a live web interface.
- **Reviewers and educators** who want a working example of verification-first LLM system design - agents whose claims are checkable by construction.

2. The three ideas behind it

Idea 1 - Specialist agents, not one giant prompt

Each job is done by a small, focused agent: a citation detective, a novelty judge, a prior-art librarian, a confidence scorer, a benchmark scout, a rebuttal writer. They run concurrently on a dependency graph with failure isolation: if one agent fails (an API hiccup, a malformed response), its lane is marked failed and everything that does not depend on it continues. You get a degraded report, never a crashed run.

Idea 2 - A knowledge graph as shared memory

Every paper is distilled into concepts, methods, datasets, claims, and results, and merged into one cross-paper graph - if five papers mention GraphRAG, that is one node with five connections. New in 0.2, each paper is also split into a **section tree** and every extracted entity is tagged with its section. That enables per-section subgraphs and lets question-answering send only the relevant slices of the relevant sections to the AI - cheaper and more precise than dumping whole papers into a prompt.

Idea 3 - Verification first

References are checked against live scholarly databases (CrossRef, then OpenAlex, with arXiv identifier lookup first), not against formatting rules. Evidence quotes are located verbatim in the source text - exact match first, then a whitespace-normalized match - and badged **verified** or **unverified**. Confidence scores are computed by a deterministic formula (no AI involved) with an explicit uncertainty band. And every run appends to a JSONL audit log, so you can always reconstruct what happened.

3. Everything you can do with it

A. Review a draft before submission (the six-lane review)

Lane	What it does	What you get
citation	Parses every bibliography entry and resolves it against live CrossRef/OpenAlex/arXiv lookups	verified / suspect / unverified per reference, with the reason (e.g. 'no matching record found' for a fabricated entry)
priorart	Retrieves related work via Semantic Scholar (OpenAlex fallback when throttled)	A ranked list of the closest prior papers
novelty	Extracts your 2-6 claimed contributions with verbatim evidence quotes; compares each claim against the retrieved prior art; then independently verifies every quote against your own text	Per-claim verdicts: novel / incremental / overlaps / anticipated, each with a checkable quote and a verified/unverified flag
confidence	Deterministic score per claim from three signals (evidence verification 1.0, novelty confidence 0.8, citation health 0.6)	A score like 0.72 +/- 0.15 - the band widens when signals disagree or are few
benchmark	Mines the graph and related work for evaluation benchmarks	Suggested benchmarks you may be expected to report on

Lane	What it does	What you get
rebuttal	Segments reviewer comments into concerns, grounds each reply in retrieved passages of your paper, drafts point-by-point responses	Replies with honesty badges: quotes verified verbatim, or flagged when no supporting quote could be checked

```
research-companion add my_draft.pdf
research-companion build # ~ $0.02 per paper
research-companion review local:XXXX --serve --report out/
```

B. Run a research lab (new in 0.2)

The bucket. `research-companion lab serve` opens a local web app. Drop in papers one at a time (arXiv ID, URL, or PDF path) or point `lab ingest <folder>` at the directory where your PDFs already live. Every pipeline stage streams events to the browser, so you **watch the knowledge graph grow in real time** - nodes bloom out of their papers as sections are parsed and entities extracted. Files that fail (scanned or corrupt PDFs) become red cards with the exact reason and a one-click Retry.

Draft alignment. Mark one paper as your draft. Every other paper then gets a verdict: which of *your* sections it **strengthens**, **challenges**, or offers an **alternative** perspective on - with a relevance meter, a rationale, and evidence quotes verified verbatim against the other paper's text. A deterministic usefulness score (high / medium / low) summarizes each paper's value to you.

Strength colors. Library cards carry a color-coded strength score computed from four explainable signals - extraction completeness (0.30), citation health (0.25), alignment with your draft (0.25), recency (0.20) - renormalized over whichever signals are available, with every signal shown. Green is strong, yellow moderate, orange weak, gray unscored, red failed.

Ask the lab. Type a question; the tool ranks all paper sections with BM25 (and, new in 0.3, optionally fuses in semantic similarity - see Section 3C), sends only the best few slices to the AI, and returns an answer with numbered citation chips. Quoted spans that cannot be found in the sources are listed in a warning panel. You can scope a question to a single section of your draft.

Compare. Pick any two papers: shared and unique concepts/methods/datasets in three columns, a side-by-side results table with the better value bolded, and an optional narrative summary.

```
research-companion lab serve # open the web workspace
research-companion lab ingest ~/papers # bulk-ingest a folder
research-companion set-draft local:XXXX # anchor your draft
research-companion align arxiv:2406.12708 # one alignment verdict
research-companion ask "which papers evaluate against human experts?"
research-companion compare arxiv:A arxiv:B
```

C. The companion layer (new in 0.3)

Home & journey. The Lab now opens on a Home view: your draft, open and addressed suggestion counts with a severity donut, a **Do this next** card driven by simple deterministic rules, and your journey - draft versions, suggestions addressed, papers added - as a timeline. First launch walks you through four steps: connect a

model, add your draft, ingest a folder, meet your suggestions.

Suggestions with revision tracking. Deterministic rules read the review, the alignments and the gap map and produce concrete, sectioned advice: discuss this paper as an alternative in related work, back this claim with evidence, fix this unverified citation. Each suggestion has a severity, a rationale and a source; they live in a docked panel behind the bell icon. When you post a revised draft, conservative matchers detect what you incorporated and flip those suggestions to **addressed** automatically - dismissals stay dismissed, and counts are tracked per version.

Converse - talk to the analysis. A floating Companion panel lets you discuss any artifact: the review, a section's alignment, a suggestion, a gap, or a paper. Answers are grounded in retrieved sections with numbered citations, quotes are verified verbatim against the sources (unverifiable spans are flagged, not hidden), and conversations persist on disk.

Timeline & gaps. A temporal view lays out concepts, methods and datasets by year. The gap overlay extracts limitations that papers admit in their own limitation/future-work sections - quoted and verified - then maps whether later papers, or your draft, address them. Open gaps relevant to your draft become suggestions.

Hybrid semantic search. With a free Hugging Face token (Settings page), Ask and Converse retrieval fuse BM25 with embedding similarity. Without the token, ranking is byte-identical to 0.2's pure BM25 - a tested degradation contract.

Saved views, themes, self-explanatory UI. Save the subgraph behind any answer as a named view and reload it from the Graph sidebar. Dark/light themes, three accents, density toggle. Every view opens with a dismissable one-line explainer, badges carry plain-English tooltips, and the Help panel documents the five core flows with a glossary.

D. Manage and mine your library

- search - typed search over papers, concepts, methods, datasets, claims, results; stats - graph statistics; view - a standalone interactive graph page.
- chat - conversational Q&A over the graph neighborhood; ask is its token-efficient, citation-checked successor.
- discover - find new papers by topic; --expand follows references/citations of what you have and surfaces the most-cited papers you are missing.
- export - Markdown, CSV, JSON, or an Obsidian vault; refcheck - the citation validator standalone; tracker/problem - library-level sweeps (new related work; problem-statement refinement against graph gaps).

4. Your first ten minutes

```
pip install research-companion # from PyPI (0.3+)

# or from source:

git clone https://github.com/Laraib-Hasan-Future/Research-Companion.git
cd Research-Companion

pip install -e "[server]"

python examples/demo_offline.py # zero keys: full review pipeline

python examples/demo_lab_offline.py # zero keys: Lab event stream
```

When you are ready for live runs, create a `.env` with your provider key (`OPENAI_API_KEY` or `ANTHROPIC_API_KEY`) and set `RESEARCH_COMPANION_PROVIDER` - or, new in 0.3, just open the Lab and enter provider and keys on the **Settings** page (keys are stored in a local `.env`, masked everywhere, and take effect without a restart). Then:

```
research-companion lab serve # browser opens at 127.0.0.1:8765
# In the UI: + Add papers -> your draft PDF -> Set as draft
# Ingest folder... -> the directory with your PDFs -> watch the graph grow
```

Real costs, measured live: building one paper's extraction costs about \$0.02 with GPT-4o; re-extracting a 16-paper library cost \$0.51; an alignment verdict or a grounded answer is one model call each. Everything cached is free to re-run - extraction, sections, and alignments are stored on disk and reused.

Everything in the web UI has a command-line twin, so the whole workflow scripts and automates cleanly.

5. Reading the outputs honestly

- **Citation verdicts.** *verified* = a live database record matches; *suspect* = title matches but authors disagree (<60% overlap) or the DOI/arXiv id conflicts; *unverified* = no plausible record exists - the fabrication case.
- **Novelty verdicts.** novel / incremental / overlaps / anticipated - always relative to *what was retrieved*. If retrieval missed the killer prior paper, the verdict cannot know about it (see Section 7).
- **0.72 +/- 0.15.** A deterministic confidence score with its uncertainty band. Wider band = fewer or more conflicting signals. A claim whose evidence quote failed verification is penalized - honesty is priced in, mathematically.
- **Verified / unverified badges.** 'Verified' means *this exact quote exists in the source text*. It does NOT mean the claim is true. 'Unverified' means the tool could not find the quote - which catches fabrication and paraphrase alike.
- **Strength colors.** A heuristic convenience for triage, not a quality judgment of the paper's science. Every signal and weight is shown so you can disagree with it.

6. Critical analysis - what it does well, with evidence

- **Fabricated-reference detection is measured, not claimed.** On a seeded corruption benchmark (fabricated titles, wrong DOIs, swapped authors; artifacts committed to the repo) the validator scores precision/recall/F1 of 1.0 - and the authors state plainly that precision is at ceiling by construction, so recall is the diagnostic number and the benchmark is a regression floor, not a field-realism claim.
- **It measures itself against reality and publishes the misses.** A live spot check on 55 real references found title-search-only resolution misflagged 32.7% of them; adding identifier-first arXiv lookup cut that to 9.1% and made resolution 3x faster. Both the before and after artifacts are committed. This measure-fix-remeasure loop is rare in research tooling.
- **The honesty machinery works end-to-end.** In live runs, unverifiable quotes actually appear flagged in the UI; alignment verdicts on unrelated papers came back 'low relevance' rather than flattering; the tool reviewed its own demo paper and penalized its confidence when three quotes failed verification.
- **Engineering quality is verifiable.** 1,576 tests (1,254 Python, 322 JavaScript), test-driven throughout, all network and AI calls behind injectable seams, a committed event-stream fixture as the executable contract between backend and frontend, and a final adversarial review pass that found and fixed real bugs before release.
- **It is genuinely local and inspectable.** Your papers never leave your machine except as prompts to the provider you chose. The store is plain JSON on disk. The audit log records every event of every run.

7. Critical analysis - limitations and failure modes, plainly

- **The novelty judge does not yet agree with human reviewers.** A pilot on 10 ICLR 2025 papers found rank agreement of $\rho = 0.28$ ($p = 0.42$) - not statistically significant. Diagnosis: the system's scores compress toward 'novel' (4.4-5.0) while humans spread 2.3-5.0. It correctly identified the genuinely novel papers but

failed to penalize weak ones when retrieval missed their true competitors. The team published this null result rather than burying it; treat novelty verdicts as a structured second opinion, not a referee.

- **Retrieval gates everything downstream.** Prior-art search uses public APIs (Semantic Scholar, OpenAlex) with keyword queries. Coverage is weakest for non-English work, non-indexed venues, and very recent papers. A novelty or alignment verdict is only as good as what retrieval surfaced.
- **Quote verification is paraphrase-blind.** A fabricated *paraphrase* of your paper passes unflagged as long as nothing is quoted. The badges verify quote fidelity, not meaning.
- **Section trees are heuristic.** Heading detection works on well-formed papers but PDF text extraction mangles some two-column layouts; the guaranteed fallback (one 'Full Text' section) keeps everything working but loses the per-section precision. Scanned PDFs without a text layer fail outright (visibly, with a retry card - but they fail).
- **The scoring weights are honest guesses.** Confidence weights (1.0/0.8/0.6), strength weights (0.30/0.25/0.25/0.20), and the alignment formula were chosen by judgment, not tuned on held-out data. They are printed on screen precisely so you can override them mentally.
- **Known AI-behavior quirks remain.** Large models are documented to be lenient reviewers; forced-JSON quirks and prompt non-compliance are caught by verification but not eliminated. Live mode costs real money and real latency (seconds per claim); extraction quality varies with the provider model.
- **Scope limits.** Extraction prompts and evaluations are CS/ML-centric today. The Lab is single-user and local (no collaboration, no cloud sync). The event recorder grows for the lifetime of a server session (fine for sessions, not for weeks of uptime). No OCR, no Zotero/Overleaf integration yet.

8. What NOT to use it for

- **Not a truth oracle.** 'Verified' badges certify that quotes exist, not that claims are correct. The tool cannot referee scientific validity.
- **Not a substitute for reading.** Alignment cards tell you where to look and why; they do not replace reading the paper that challenges your Section 4.
- **Not a ghostwriter.** It critiques, aligns, and drafts grounded rebuttal replies; it does not write paper content. Venue AI-assistance policies remain the author's responsibility - disclose use where required.
- **Not a plagiarism or compliance checker.** Mature commercial tools own that space; this tool deliberately does the things they do not.

9. Under the hood (for the curious)

An in-process async event bus connects everything. Agents are plain Python classes with a name, a role, dependencies, and an async run method; the orchestrator topologically sorts them and runs whatever is ready, concurrently. Every event (started, finding, done, error, plus nine Lab events like `graph_delta` and `alignment_ready`) is a typed object serialized identically to the dashboard, the audit log, and the Lab's server-sent-event stream - one contract, tested against a committed fixture. All AI and network calls sit behind injectable seams, which is why the entire pipeline runs in the test suite with zero network. The web frontend is deliberately boring: no build step, no framework - plain ES modules plus a vendored graph-rendering library, so ``pip install`` is the whole story.

10. Getting it

Repository (MIT): <https://github.com/Laraib-Hasan-Future/Research-Companion>

```
pip install research-companion # PyPI, full install
research-companion lab serve # the web workspace
python examples/demo_offline.py # see it work with zero keys
```

Version 0.3.0. 1,576 tests. Local-first. Every verdict checkable. If you find a place where the tool claims more than it can verify - that is a bug; please report it.